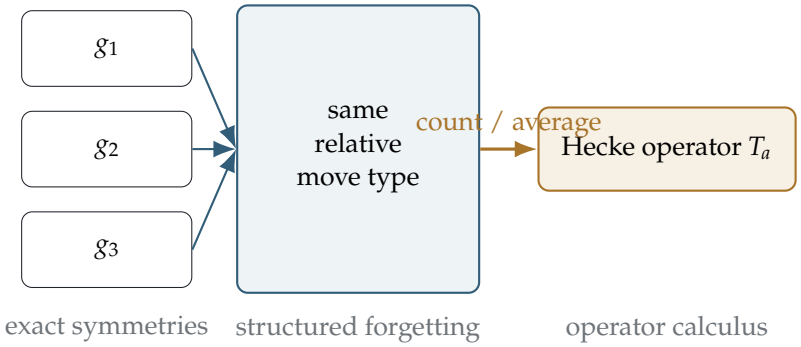
HECKE ALGEBRAS

How Symmetry Becomes Geometry, Counting, and Operators

A visual guide from double cosets to structured moves



Mirco A. Mannucci and Robin Langer

Revised booklet edition – October 2026

Groups describe exact motion. Hecke algebras describe counted relative motion through structured space.

Contents

Preface: the anti-mystification **2**

1 Sharp symmetry and blurred symmetry **2**

2 The palace: relative position instead of exact motion **3**

3 A complete finite example: S_3 becomes a graph **4**

4 Convolution: multiplication by counting paths **5**

5 Where q comes from: geometry with thickness **6**

6 A worked $q = 2$ micro-example **7**

7 Flags: permutations become relative positions **8**

8 Buildings and trees: the geometry behind the algebra **8**

9 Lattices and arithmetic correspondences **9**

10 Modular forms: eigenfunctions of arithmetic averaging **10**

11 Spectra and random walks: why eigenvalues keep appearing **10**

12 Hecke algebras as symmetry-respecting compression **11**

13 Which Hecke algebra? A map of the family **12**

14 From similarity to structured moves **13**

15 A tiny computational laboratory **13**

16 What is genuinely Hecke-like—and what is not **15**

17 A compact field guide **16**

18 The shortest useful mental model **16**

A Three exercises worth actually doing **18**

B Common misunderstandings **18**

C A "where would I meet this?" field guide **18**

D Further reading **19**

Preface: the anti-mystification

Hecke algebras have an unfortunate pedagogical reputation. They are often first met after a long sequence of prerequisites—Coxeter groups, modular forms, p-adic groups, representation theory—and by then the notation is dense enough that the basic idea disappears.

This booklet takes the opposite route. We begin with one elementary question:

The question

What happens if a symmetry problem contains more exact information than we actually care about?

A group remembers exact motions. But in geometry, arithmetic, data analysis, and computation we frequently care only about a *type* of motion: move to a neighboring chamber, replace a lattice by one of index p, place one flag in a given relative position to another, or transform a document by a certain admissible operation. Once exact motion is deliberately blurred, composition stops being deterministic. One coarse move followed by another can have several possible coarse outcomes, with multiplicities.

That is the entry point to Hecke theory.

The guiding progression will be

$$\boxed{\text{exact symmetry}} \longrightarrow \boxed{\text{relative position}} \longrightarrow \boxed{\text{correspondence}} \longrightarrow \boxed{\text{counted composition}} \longrightarrow \boxed{\text{operator}}.$$

The aim is not to avoid formulas. It is to make every formula earn its place. We will compute a complete finite example, draw the corresponding graph, see where the parameter q comes from, connect the same mechanism to lattices and modular forms, and finally ask why an AI engineer might care about an algebra of *moves* rather than only a geometry of *embeddings*.

The final AI section is deliberately cautious. Ordinary agent workflows are not automatically Hecke algebras. The useful claim is narrower: Hecke theory gives a precise vocabulary for quotienting irrelevant detail, representing typed correspondences, composing them by summing over intermediate states, and studying the resulting operators. That is enough to motivate a computational laboratory without pretending that an analogy is a theorem.

A reader comfortable with basic linear algebra and the idea of a group should be able to follow the main line. More advanced objects are introduced only far enough to expose the architecture.

1. Sharp symmetry and blurred symmetry

A group is a machine of reversible exact motions. If a square is rotated by 90° and then reflected, there is one exact resulting symmetry. If two invertible matrices are multiplied, the product is one invertible matrix. Group multiplication is deterministic:

$$g \cdot h = gh.$$

The group algebra $\mathbb{C}[G]$ allows formal linear combinations

$$a_1 g_1 + \cdots + a_n g_n,$$

but its basis still remembers every exact element of G.

A Hecke algebra begins when we decide that some distinctions are irrelevant. Let $H \leq G$ be the subgroup of symmetries we agree to ignore. Instead of an individual g, we consider the packet

$$HgH = \{h_1 g h_2 : h_1, h_2 \in H\}.$$

This is a *double coset*. The left copy of H forgets internal coordinates at the source; the right copy forgets them at the target.

First slogan

A group algebra remembers sharp symmetry. A Hecke algebra remembers symmetry after a controlled blur.

The blur has two extremes. If $H = \{e\}$, then $HgH = \{g\}$ and nothing has been forgotten: we recover the group algebra. If $H = G$, every element belongs to the same double coset and almost everything has been forgotten. Interesting Hecke algebras live between those extremes.

1.1 A visual model of structured forgetting

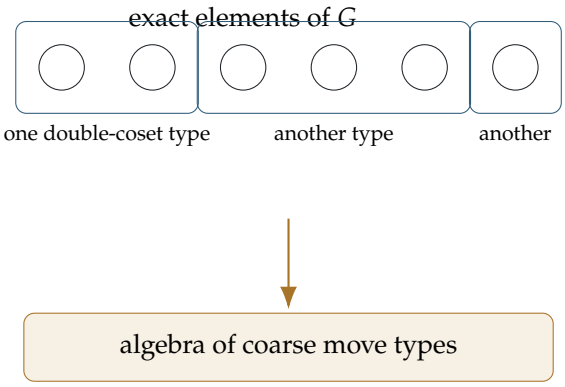


Figure 1: The quotient does not forget randomly. It groups exact motions into invariant move types.

This is why the phrase *structured forgetting* is useful. The purpose is not compression for its own sake. We deliberately erase distinctions that do not affect the question and preserve the way the surviving classes compose.

2. The palace: relative position instead of exact motion

Imagine a palace with many rooms. A huge symmetry group moves you around while preserving the architecture. But suppose you do not care about your precise location inside a room. You care only about relations such as

- same room type,
- adjacent room,
- two doors away,
- same wing,
- opposite orientation.

Then the fundamental object is no longer an exact motion. It is a *relative-position type*.

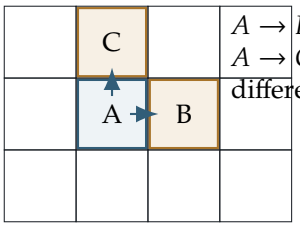


Figure 2: The palace analogy: many exact moves can represent the same invariant relation.

If G acts transitively on a set X and H stabilizes a reference point x_0, then

$$X \cong G/H.$$

The invariant relative positions of pairs of points are classified by

$$H\backslash G/H.$$

That is the geometric meaning of double cosets.

Why both left and right?

A right coset gH forgets a change of coordinates after the move. A left coset Hg forgets a change before it. A double coset HgH forgets both, exactly what we need when source and target objects have internal coordinate freedom.

This simple observation explains why double cosets occur in flag varieties, lattices, buildings, symmetric spaces, and arithmetic correspondences. They are not bookkeeping. They are a classification of relative position.

3. A complete finite example: S_3 becomes a graph

The best way to demystify a Hecke algebra is to calculate one from beginning to end.

Let

$$G = S_3, \qquad H = \langle (12) \rangle = \{e, (12)\}.$$

There are only two double cosets. One is H itself. If $g = (13)$, the other is

$$D = H(13)H,$$

which contains the remaining four permutations. The resulting Hecke algebra is therefore two-dimensional.

Use normalized convolution so that the characteristic function 1_H is the identity. Write $T = 1_D$. A direct count gives

$$\boxed{T^2 = T + 2I.}$$

That tiny equation contains the whole mechanism: a coarse move followed by the same coarse move can return to the starting type in two ways and end in the nontrivial type in one way.

3.1 The same calculation as a graph

The quotient G/H has three points. The nontrivial relation is simply "different point." Therefore T is represented by the adjacency matrix of the triangle K_3:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Then

$$A^2 = A + 2I.$$

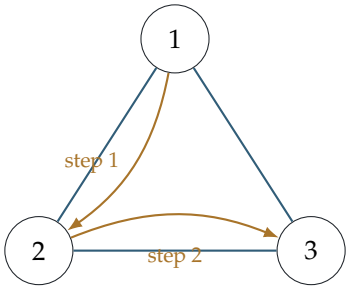


Figure 3: The S_3/H Hecke operator is just adjacency on a triangle. Two-step path counting gives the multiplication rule.

The eigenvalues are 2 and -1. So before we have said anything about modular forms or p-adic groups, counting geometry has already produced a spectrum.

Engineer translation

If a finite state space has a small number of symmetry-invariant relations, each relation gives a matrix. Closing those matrices under composition produces an operator algebra. Hecke theory is a highly structured version of this idea in which the relations come from symmetry rather than arbitrary graph design.

4. Convolution: multiplication by counting paths

Why does the product look like path counting? Because the natural multiplication is convolution.

For a finite group G, define

$$(f * k)(g) = \sum_{x \in G} f(x)k(x^{-1}g).$$

If f and k are bi-H-invariant, then so is $f * k$. Therefore the space of functions constant on double cosets is closed under convolution.

For basis elements $T_a = 1_{HgaH}$ we obtain

$$T_a T_b = \sum_c N_{ab}^{\,c} T_c.$$

The coefficient $N_{ab}^{\,c}$ counts compatible intermediate states.

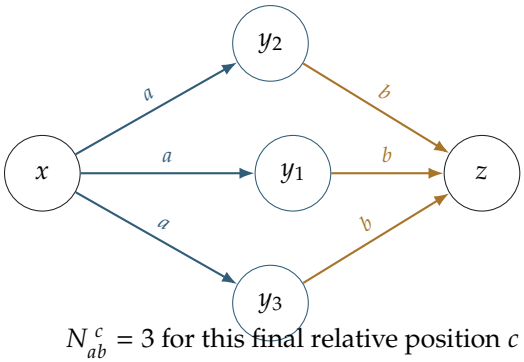


Figure 4: Convolution sums over the intermediate object. Multiplication constants are path counts or their measured analogues.

This is the algebraic heart of the subject. In finite groups, the sum is literal counting. In locally compact groups, sums become integrals against Haar measure. In algebraic geometry, counting points becomes push-pull along correspondences. In categorified versions, coefficients are replaced by richer homological objects. The same skeleton persists.

4.1 Kernel viewpoint

A matrix is a finite kernel $K(x, y)$. Matrix multiplication says

$$(K_1K_2)(x, z) = \sum_y K_1(x, y)K_2(y, z).$$

The Hecke convolution formula is the symmetry-adapted cousin of this familiar identity. This is one reason the subject should feel less alien to engineers than its notation suggests.

5. Where q comes from: geometry with thickness

One often hears that a Hecke algebra is a "q-deformation" of a group algebra. That is useful, but pedagogically backwards. The parameter q is frequently present because the geometry has q-dependent numbers of choices.

Consider the finite field $\mathbb{F}_q$. The projective line

$$\mathbb{P}^1(\mathbb{F}_q)$$

has exactly $q + 1$ points. Why? A line through the origin in $\mathbb{F}_q^2$ is determined by a slope $m \in \mathbb{F}_q$, plus one vertical line.

Now move from lines to complete flags. For $G = GL_n(\mathbb{F}_q)$ and B the upper-triangular subgroup, the double cosets

$$B\backslash G/B$$

are indexed by permutations $w \in S_n$. The symmetric group gives the *types* of relative position. The field size q controls how many actual flags realize those types.

For a simple reflection, one standard Hecke relation is

$$(T_s - q)(T_s + 1) = 0,$$

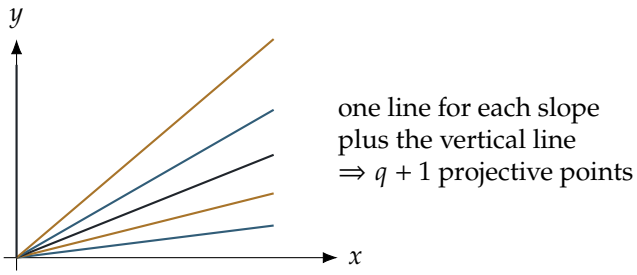


Figure 5: Finite-field geometry has real combinatorial thickness. The parameter q often counts that thickness.

or equivalently

$$T_s^2 = (q-1)T_s + qI.$$

At $q = 1$ this collapses toward $s^2 = 1$.

The geometric slogan

The Weyl group tells you which moves exist. The Hecke algebra tells you how thickly those moves occur.

This is much more informative than saying that q is merely a formal deformation parameter. It can later be treated formally, but its geometric origin is often literal counting.

6. A worked $q = 2$ micro-example

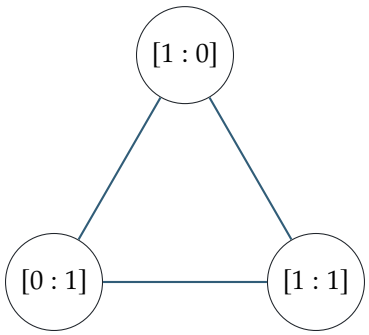
The smallest finite field already makes the geometry tangible. Let $V = \mathbb{F}_2^2$. Its nonzero vectors are

$$(1,0), \qquad (0,1), \qquad (1,1).$$

Because the only nonzero scalar in $\mathbb{F}_2$ is 1, each nonzero vector spans a different one-dimensional subspace. Hence

$$|\mathbb{P}^1(\mathbb{F}_2)| = 3.$$

So the projective line over $\mathbb{F}_2$ is literally a three-point space.



three lines through the origin $\Longleftrightarrow$ three projective points

Figure 6: $\mathbb{P}^1(\mathbb{F}_2)$ already gives the triangle geometry from the S_3 example.

This is not a coincidence. The group $PGL_2(\mathbb{F}_2)$ acts on these three projective points and is isomorphic to S_3. The rank-one finite geometry and the earlier permutation example are two descriptions of the same tiny homogeneous space.

For a simple Hecke generator at $q = 2$, the quadratic relation becomes

$$(T - 2)(T + 1) = 0, \qquad \text{so} \qquad T^2 = T + 2I.$$

That is exactly the triangle relation we already computed.

Why this example matters

The abstract-looking q-deformation is already visible in the smallest possible case. At $q = 2$, "finite-field thickness" and "two-step path counting on a triangle" are literally the same calculation.

7. Flags: permutations become relative positions

A complete flag in an n-dimensional vector space is a chain

$$0 \subset V_1 \subset V_2 \subset \cdots \subset V_{n-1} \subset V, \qquad \dim V_i = i.$$

Fixing one reference flag, the subgroup that preserves it is a Borel subgroup B. The surprising fact is that the relative position of two complete flags is encoded by a permutation.

In dimension three, there are six possible relative-position types because S_3 has six elements. The geometry may contain many actual flags, especially over $\mathbb{F}_q$, but the *types* are few.

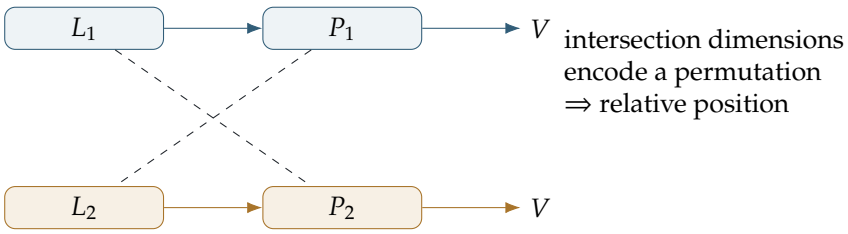


Figure 7: Two flags have internal coordinates, but their invariant incidence pattern is discrete.

A Hecke operator now has an intuitive meaning: from a flag F, sum over all flags F' in one prescribed relative position. Applying two such operators means making two relative moves and counting intermediate flags.

This is the bridge from permutations to geometry. The algebra is not an abstract deformation invented for its own sake. It is the multiplication table of incidence relations among flags.

8. Buildings and trees: the geometry behind the algebra

A building is a highly symmetric chamber complex whose apartments look like Coxeter complexes. For our purposes, the important point is simple: buildings turn abstract Weyl-group move types into actual geometry with branching.

The rank-one model is a regular tree. For $PGL_2(\mathbb{Q}_p)$, the Bruhat–Tits building is a $(p + 1)$-regular tree. Vertices can be represented by homothety classes of $\mathbb{Z}_p$-lattices in $\mathbb{Q}_p^2$. Each vertex has $p + 1$

neighbors because

$$|\mathbb{P}^1(\mathbb{F}_p)| = p + 1.$$

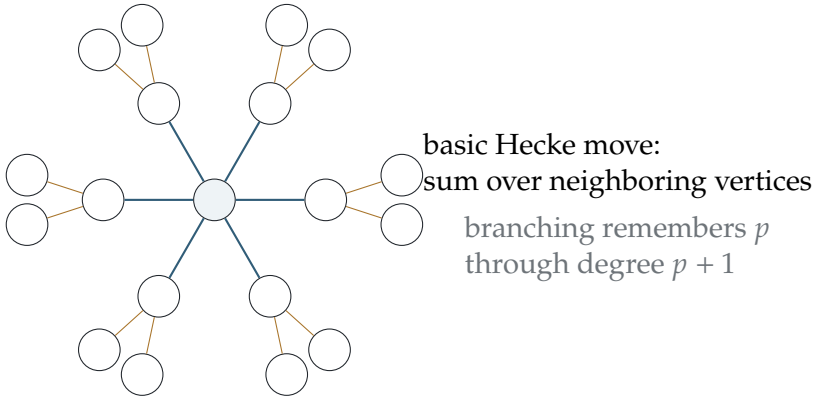


Figure 8: A building makes "q-deformation" visible as branching geometry.

The adjacency operator on this tree is a Hecke operator. Its spectrum controls random walks and harmonic analysis. Higher-rank buildings replace scalar distance by Weyl-valued relative position and replace one type of edge by several panel types.

So the same sequence keeps reappearing:

$$\text{symmetry} \to \text{quotient geometry} \to \text{relative moves} \to \text{adjacency operators} \to \text{spectrum}.$$

9. Lattices and arithmetic correspondences

The lattice picture gives one of the most concrete bridges to number theory.

A lattice in $\mathbb{R}^2$ is a discrete grid generated by two independent vectors. Hecke operators can often be viewed as summing over lattices related by finite-index inclusion. A degree-p move sends a lattice L to the formal sum of suitable sublattices $L' \subset L$ with

$$[L : L'] = p.$$

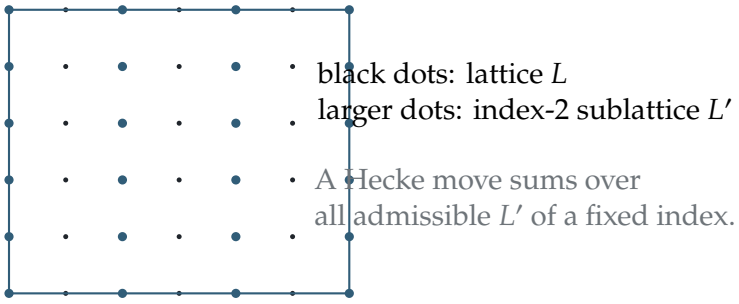


Figure 9: Arithmetic Hecke operators can be read as averaging over finite-index modifications of lattices.

Composition then means: choose a sublattice of one index, then another, and count how the final lattice can arise. Again we have objects, structured modifications, summation over alternatives, and composition by counting.

This perspective is the conceptual bridge to classical Hecke operators on modular forms, because lattices in the complex plane and elliptic curves are deeply intertwined.

10. Modular forms: eigenfunctions of arithmetic averaging

A modular form is a highly constrained analytic function on the upper half-plane with a transformation law under a discrete group such as $SL_2(\mathbb{Z})$. Hecke operators act on modular forms by averaging over arithmetic correspondences.

For a prime p, a classical Hecke operator can be written schematically as

$$(T_p f)(z) = p^{k-1} f(pz) + \frac{1}{p} \sum_{a=0}^{p-1} f\left(\frac{z+a}{p}\right),$$

with normalization conventions varying across sources.

The formula looks analytic, but the mechanism is still the same: do not choose one arithmetic modification; sum over all modifications of a given type.

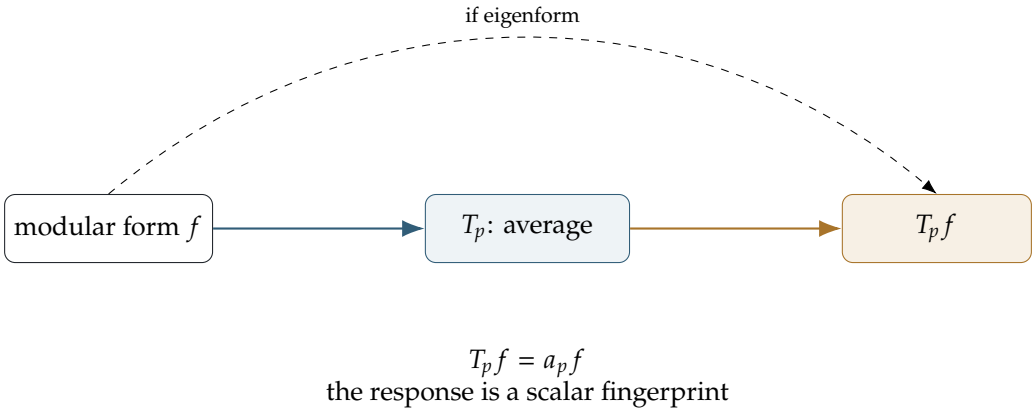


Figure 10: A Hecke eigenform is aligned with the arithmetic averaging operators.

If f is a simultaneous eigenform,

$$T_n f = a_n f,$$

then the eigenvalues a_n encode remarkable arithmetic information. They enter Euler products, connect to L-functions, and in the Langlands picture become local spectral data.

Why operators matter

A Hecke algebra is not valuable merely because it has a multiplication table. Its elements act. Once they act, spectra and eigenobjects become available. Geometry becomes harmonic analysis.

11. Spectra and random walks: why eigenvalues keep appearing

Once a Hecke element is realized as an adjacency or averaging operator, spectral questions are unavoidable. The simplest example is again the triangle:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

The constant vector

$$\mathbf{1} = (1, 1, 1)^T$$

is an eigenvector with eigenvalue 2: from every vertex there are two neighbors. Any vector whose coordinates sum to zero lies in the orthogonal two-dimensional subspace, where the eigenvalue is -1.

Thus

$$\mathrm{Spec}(A) = \{2, -1, -1\}.$$

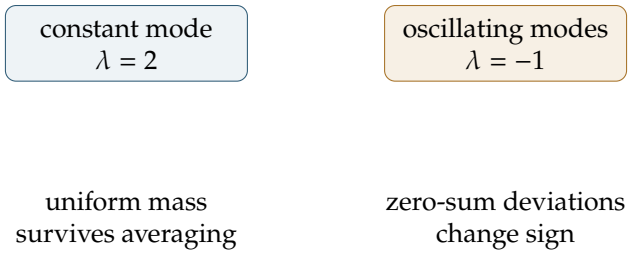


Figure 11: Even the smallest Hecke example splits the state space into harmonic modes.

For a random walk, normalize adjacency by the vertex degree. Then the top eigenvalue becomes 1, corresponding to the stationary uniform mode, while the remaining eigenvalues govern how fast deviations decay or oscillate.

This elementary fact scales dramatically. On buildings, spherical functions are simultaneous eigenfunctions of commuting Hecke operators. On modular forms, Hecke eigenvalues become arithmetic fingerprints. In expander theory, spectral bounds on Hecke-type graphs control mixing. The vocabulary changes, but the pattern does not:

$$\text{counted geometry} \longrightarrow \text{operators} \longrightarrow \text{eigenmodes}.$$

Engineer translation

If your structured moves define operators, eigenvectors can reveal stable workflow modes, bottlenecks, recurrent patterns, or invariant subspaces. The spectral step is where an operator calculus can become predictive rather than merely descriptive.

12. Hecke algebras as symmetry-respecting compression

The full group can be enormous. Yet a problem may only observe H-invariant information. In that case, exact group elements contain redundant detail.

For a finite pair (G, H), the Hecke algebra can be viewed—up to left/right convention—as

$$\mathcal{H}(G, H) \cong \mathrm{End}_G\big(\mathbb{C}[G/H]\big).$$

The right-hand side means: all linear operators on the quotient geometry that commute with the original G-symmetry.

This is a powerful form of compression. It is not numerical dimensionality reduction. We do not ask an optimization algorithm to guess what can be discarded. The subgroup H explicitly states which distinctions are irrelevant, and equivariance tells us which operators survive.

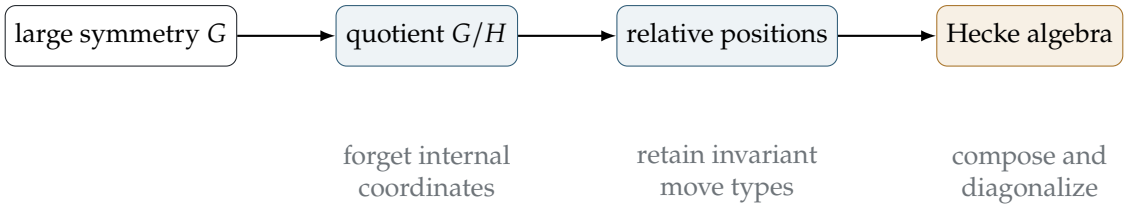


Figure 12: Hecke theory is compression with structural guarantees.

For machine learning readers, this suggests an important distinction. An embedding compresses objects by placing them near similar objects. A Hecke-style quotient compresses a transformation problem by identifying states or motions that are equivalent for a task. Those are different operations.

13. Which Hecke algebra? A map of the family

One source of needless confusion is the phrase "the Hecke algebra." There is no single universal object with that name. Several constructions share the same architecture while living in different settings.

Setting	**Typical object**	**What the basis means**
Finite pair (G, H)	bi-H-invariant functions on G	double-coset / relative-position types
Coxeter system (W, S)	Iwahori–Hecke algebra	deformed Weyl-group moves T_w
p-adic group (G, K)	spherical Hecke algebra	compact-double-coset modification types
Modular forms	commuting Hecke operators T_n	arithmetic correspondences of degree n
Flag geometry	adjacency / convolution algebra	relative positions of flags
Categorical geometry	Hecke category	sheaves or bimodules replacing basis vectors

The family resemblance can be summarized by four words:

symmetry quotient correspondence convolution

.

Historically, Erich Hecke introduced operators in the theory of modular forms because they exposed arithmetic structure invisible to the transformation law alone. Later developments showed that the same operator logic belongs to a much larger representation-theoretic and geometric world.

This is why it is safer to ask

Which Hecke algebra, attached to which symmetry situation?

than to search for one definition that hides all distinctions.

14. From similarity to structured moves

Modern AI systems are extraordinarily good at representing *objects*. Embeddings map text, images, code, or other data into vector spaces where similarity becomes geometric proximity:

$$\text{object} \longrightarrow \text{vector} \longrightarrow \text{nearest neighbors.}$$

That answers a valuable question:

What is close to what?

But many systems also need to reason about transformations:

What can be done from here, what is admissible, and how do operations compose?

A document can be summarized, criticized, risk-extracted, rewritten, translated, or audited. A proof attempt can be formalized, checked, normalized, or searched for counterexamples. An agent state can be searched, compressed, generated from, tested, and audited.

Those operations are not merely similarities between states. They are directed correspondences.

14.1 Order is information

If G means "generate" and T means "test", then

$$TG \neq GT$$

in a realistic workflow. Generate-then-test and test-then-generate are different procedures, often with different admissibility and different outcomes.

Noncommutativity is therefore not an inconvenience. It can encode workflow order effects.

What Hecke theory contributes

The useful import is not the word "Hecke." It is a checklist: identify task-relevant equivalence, define invariant move types, represent them as correspondences, compose through intermediate states, inspect structure constants, test associativity, and study spectra or invariants when they exist.

This is exactly where a small computational companion becomes useful.

15. A tiny computational laboratory

The companion repository **Hecke-Lab** starts with the simplest executable object: a finite weighted relation. A state set is finite, an operator stores weighted transitions, and composition counts two-step paths.

In the current API, if A and B are operators, the expression `A @ B` means "first B, then A." A minimal example looks like this:

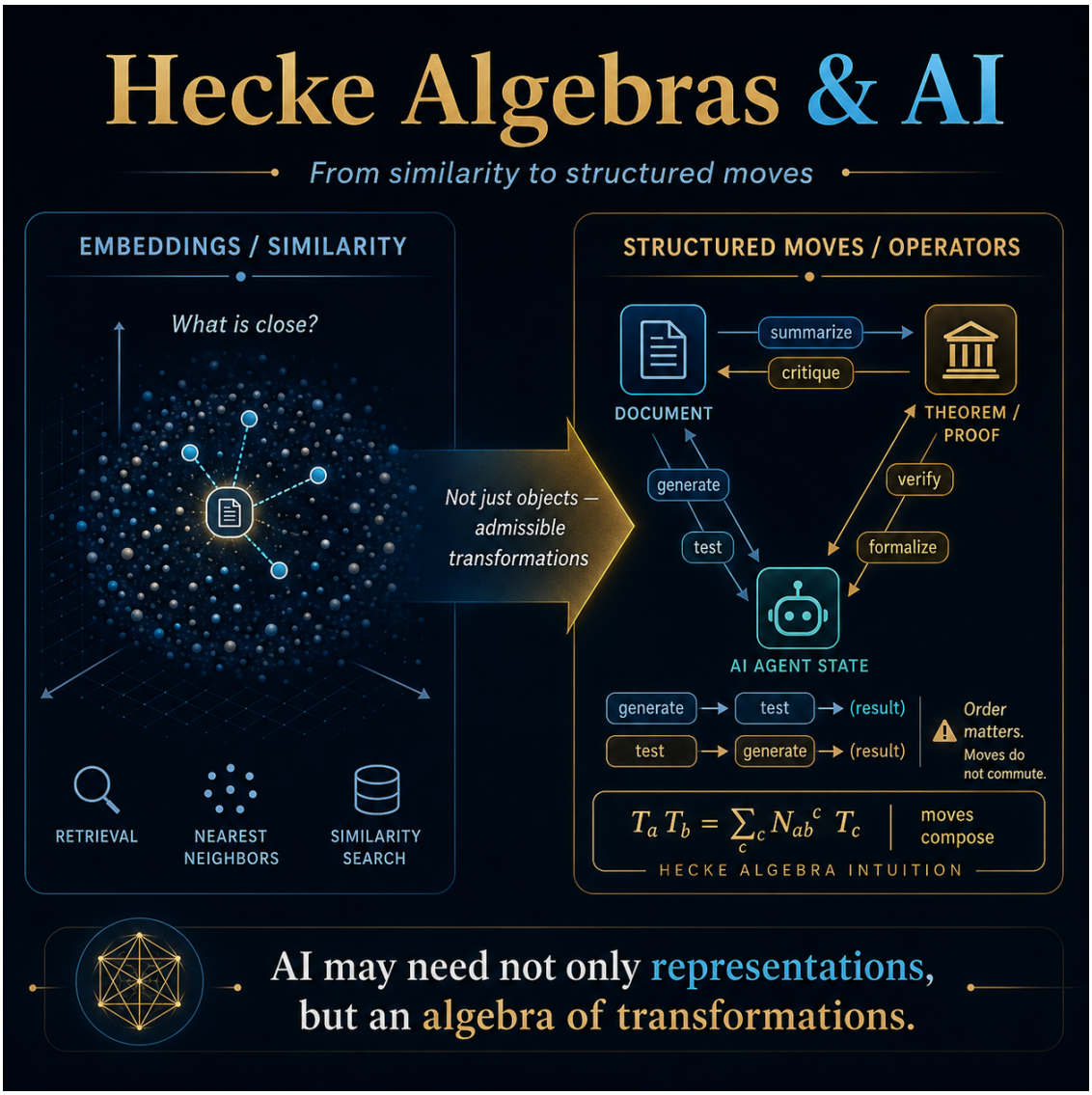


Figure 13: The companion Hecke-Lab idea: complement object similarity with an explicit calculus of admissible moves.

```python
from hecke_lab import FiniteOperator

states = ("question", "draft", "tested")

generate = FiniteOperator.from_edges(
    "generate", states,
    [("question", "draft")]
)

test = FiniteOperator.from_edges(
    "test", states,
    [("draft", "tested")]
)

pipeline = test @ generate   # first generate, then test
print(pipeline.pretty())
```

The mathematics is just matrix/kernel composition:

$$(T_a T_b)(x, z) = \sum_y T_b(x, y) T_a(y, z).$$

That is deliberately modest. A generic relation algebra is not automatically a Hecke algebra. The research question is whether a useful quotient and basis of move types can make real workflow composition *stable enough* to acquire additional algebraic structure.

15.1 The right experimental question

Instead of asking "Can AI workflows be called Hecke algebras?", ask:

Empirical research question

Can we discover a quotient of workflow states on which typed move composition becomes sparse, stable, and approximately algebraic?

That question is testable. Given workflow traces, estimate empirical structure constants

$$N_{ab}^{c} \approx \text{frequency or weight of } a \text{ followed by } b \text{ behaving like } c.$$

Then measure:

- closure: do products remain in the chosen move vocabulary?
- associativity error: does $(T_a T_b)T_c$ agree with $T_a(T_b T_c)$?
- sparsity: are most structure constants zero or negligible?
- stability: do the constants persist across tasks or contexts?
- noncommutativity: which order effects are systematic?
- spectra: do operator eigenmodes predict successful or failed workflows?

Failure is informative. Large associativity error may mean the state quotient is too coarse, context has been omitted, or the move vocabulary is not closed.

16. What is genuinely Hecke-like—and what is not

The analogy becomes meaningful only when there is structure behind it. A generic matrix algebra is not a Hecke algebra simply because matrices compose.

A serious Hecke-style model should identify several of the following ingredients:

1. a principled equivalence relation or stabilizer;
2. move types defined by invariant relative position;
3. composition coefficients obtained from counting, measure, geometry, or data;
4. a distinguished basis of move types;
5. an involution or reverse correspondence;
6. admissibility constraints or invariants;
7. representation-theoretic or spectral consequences.

Bad analogy

Take six arbitrary workflow matrices and give their span a multiplication table. That is an operator algebra, but there is no reason to call it Hecke-like.

Stronger analogy

Start from a state space with known nuisance symmetries, quotient them out, classify invariant transition types, estimate their composition from traces, verify approximate closure and associativity, and discover stable eigenmodes correlated with workflow outcomes. Now the Hecke comparison has mathematical content.

This distinction matters because Hecke theory is powerful precisely because its coefficients are not arbitrary. They are constrained by geometry and symmetry.

17. A compact field guide

Object	Working intuition
Group G	The full world of exact symmetries or reversible motions.
Subgroup H	The internal ambiguity we deliberately ignore; often the stabilizer of a reference object.
Coset G/H	The space of objects obtained by moving the reference object.
Double coset HgH	An invariant relative position, or a coarse type of move.
Hecke operator	Sum or average over all targets in one relative-position type.
Convolution	Composition by summing over intermediate objects.
Structure constant N_{ab}^{c}	Number, measure, or weight of ways move a followed by b produces type c.
Parameter q	Often the size of a finite field, hence a literal count of geometric choices.
Eigenobject	An object whose response to Hecke averaging is scalar or otherwise spectrally simple.
Building	A geometry in which Weyl-group move types acquire thickness and branching.
Hecke-Lab	A computational sandbox for testing whether structured move composition is useful beyond the classical setting.

18. The shortest useful mental model

We can now compress the booklet into one diagram:



Figure 14: The master mechanism.

A group answers:

Where do I land after this exact motion?

A Hecke algebra asks:

What type of place can I land in after this type of motion, and how many ways can I get there?

That is why Hecke algebras appear wherever symmetry, geometry, incidence, and counting meet.

For classical mathematics, they organize flags, buildings, lattices, modular forms, automorphic representations, and much more. For computation, their broader lesson is methodological: sometimes the right primitive is neither an object nor an exact action, but a *typed relative move*; and sometimes the right model of composition is not deterministic function composition, but convolution over admissible intermediate states.

Final slogan

Groups describe exact motion. Hecke algebras describe counted motion through structured space.

A. Three exercises worth actually doing

A.1 Reproduce the triangle relation

Take

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Compute A^2 directly and verify $A^2 = A + 2I$. Then explain each matrix entry as a two-step path count.

A.2 Count projective points

List all one-dimensional subspaces of $\mathbb{F}_2^2$ and $\mathbb{F}_3^2$. Verify that the counts are 3 and 4, respectively, and connect them to the formula $|\mathbb{P}^1(\mathbb{F}_q)| = q + 1$.

A.3 Design a workflow quotient

Choose a small AI workflow with at most eight states. Define an equivalence relation that forgets one nuisance distinction, such as formatting, model identity, or provenance detail. Propose three move types on the quotient. Ask whether their compositions close. If not, decide whether the move vocabulary or the quotient needs refinement.

B. Common misunderstandings

"A Hecke algebra is just a deformed group algebra." Sometimes, especially for Iwahori–Hecke algebras of Coxeter systems. But the broader mechanism is convolution of invariant correspondences or double cosets.

"The parameter q is formal decoration." It can be treated formally, but in many foundational examples it literally counts finite-field choices.

"Double cosets are merely notation." They encode invariant relative position. In many geometric examples, that is the central object.

"Any workflow operator algebra is Hecke." No. The analogy is strongest when move classes come from principled quotient structure and composition coefficients are constrained rather than invented.

"Hecke operators are only number-theoretic." Classical number theory made them famous, but the same convolution mechanism appears in representation theory, finite geometry, buildings, harmonic analysis, and categorification.

C. A "where would I meet this?" field guide

If you work on...	**The Hecke-shaped question is...**
Finite groups	What invariant relative positions remain after fixing a stabilizer, and how do their incidence relations compose?
Combinatorics	What adjacency relations form a closed algebra, and what do their spectra say about the underlying geometry?
Finite geometry	How many flags, subspaces, or chambers realize a prescribed sequence of relative moves?
Representation theory	Can a large group representation be probed through the smaller algebra acting on H-fixed vectors?
Number theory	Which arithmetic correspondences act on modular or automorphic forms, and what do their eigenvalues encode?
p-adic geometry	What does averaging over compact double cosets reveal about a representation or a building?
Random walks / expanders	Which symmetry-generated adjacency operators control mixing and expansion?
AI systems	Can task-relevant equivalence classes and typed transformations support a stable operator calculus richer than similarity search?
Workflow analysis	Which order effects are systematic, which move compositions close, and which invariants survive repeated transformations?

The point of the table is not that every problem above "is Hecke theory." It is that Hecke theory trains a useful habit of thought: when exact state descriptions are too detailed, look for invariant relation types and study how they compose.

D. Further reading

The following references lead from the level of this booklet toward the standard theory.

- A. Björner and F. Brenti, *Combinatorics of Coxeter Groups*. Springer, 2005.
- D. Bump, *Automorphic Forms and Representations*. Cambridge University Press, 1997.
- R. Carter, *Finite Groups of Lie Type*. Wiley, 1985.
- F. Diamond and J. Shurman, *A First Course in Modular Forms*. Springer, 2005.
- J. E. Humphreys, *Reflection Groups and Coxeter Groups*. Cambridge University Press, 1990.
- N. Iwahori and H. Matsumoto, "On some Bruhat decomposition and the structure of the Hecke rings of p-adic Chevalley groups," *Publ. Math. IHÉS* 25 (1965).
- D. Kazhdan and G. Lusztig, "Representations of Coxeter groups and Hecke algebras," *Invent. Math.* 53 (1979).
- I. G. Macdonald, *Spherical Functions on a Group of p-adic Type*. Ramanujan Institute, 1971.
- J.-P. Serre, *Trees*. Springer, 1980.

Companion project: Hecke-Lab — finite experiments with composable structured moves.
This booklet intentionally stops where the laboratory begins.